

Reviewer Pack — Noncommutative L^p Geometric Entropy Lower Bound for Communi...

Entry 1e09a50da5e3 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-23 11:37:36 UTC

Noncommutative L^p Geometric Entropy Lower Bound for Communication Complexity

Entry ID: 1e09a50da5e3 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-23 11:37:36 UTC

1. Conjecture

Field A (mathematical branch): Noncommutative Geometry and Algebra (L^p Geometry) **Field B** (complexity object): Communication Complexity (Disjointness)

Statement:

[‘For any partial function $f: \{0,1\}^n \rightarrow \{0,1\}$, the noncommutative L^p geometric entropy $H_p(f)$ of f satisfies $H_p(f) = \Omega(n^{(1-p/(p+1))})$ for all $p \in [0, 1]$.’, ‘This implies that the randomized communication complexity of f is lower bounded by $\Omega(n^{(1-1/p)})$ when both parties have access to a shared L^p geometric description of f .’, ‘For disjointness, this yields a communication complexity of $\Omega(n)$ as expected.’]

Rationale (proposer’s reasoning):

[“ L^p geometric entropy has been used in noncommutative geometry to measure the complexity of functions and distributions. It is believed that higher values of p correspond to more ‘spread-out’ or ‘complex’ functions.”, ‘If the communication complexity of a function can be lower bounded by its L^p geometric entropy, it would provide a new approach to understanding the complexity of disjointness and other communication problems.’, ‘This conjecture builds on the idea that the complexity of a function can be characterized by its geometric properties, which is a theme present in both noncommutative geometry and communication complexity.’]

Taxonomy category: COMM_DISJ (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): f22bbc366edfd1f2

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

To support the conjecture, the computed L^p geometric entropy $H_p(f)$ must be at least $\Omega(n^{(1-p/(p+1))})$ for all $p \in [0, 1]$ with a p -value of ≤ 0.05 after analyzing at least 100 random functions for each n and p .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.95	SAFE	SAFE
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "noncommutative geometry L^p " AND "communication complexity disjointness" - " L^p geometric entropy" AND "communication complexity lower bound" - "shared L^p geometric description" AND "randomized communication complexity"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def communication_complexity(f):
        n = len(f)
        if n == 1:
            return 0
        matrix = [[0] * (n + 2) for _ in range(n + 2)]
        for x, y in f.keys():
            index = x * (n + 1) + y
            matrix[index][index + 1] += 1
        # Compute the trace of the matrix
        trace = sum(matrix[i][i] for i in range(n + 2))
        return trace

    def l_p_geometric_entropy(f, p):
        n = len(f)
        if n == 1:
            return 0
        matrix = [[0] * (n + 2) for _ in range(n + 2)]
        for x, y in f.keys():
            index = x * (n + 1) + y
            matrix[index][index + 1] += 1
        # Compute the trace of the p-th power of the matrix
        trace_p = sum(matrix[i][i]**p for i in range(n + 2))
        return trace_p
```

```

n_values = [5, 10, 15, 20, 30, 40]
results = []

for n in n_values:
    f = {tuple(random.sample(range(2), n)): random.choice([0, 1]) for _ in range(n * (n + 1))}
    entropy = communication_complexity(f)
    p_values = [Fraction(i, 10) for i in range(11)]
    for p in p_values:
        H_p = l_p_geometric_entropy(f, p)
        results.append((n, p, H_p))

metric_value = sum(H_p * n**((1 - p) / (p + 1)) for n, p, H_p in results) / len(results)
instances_tested = len(results)
conjecture_holds = all(H_p >= n**((1 - p) / (p + 1)) for _, p, H_p in results)
counterexample = "" if conjecture_holds else "mapping_undefined"

return {
    "metric_name": "communication_complexity",
    "metric_value": metric_value,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17, 19]
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_da040e2b.py", line 75, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_da040e2b.py", line 49, in run_trial
    f = {tuple(random.sample(range(2), n)): random.choice([0, 1]) for _ in range(n * (n + 1))}
             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "/usr/lib/python3.12/random.py", line 430, in sample
    raise ValueError("Sample larger than population or is negative")
ValueError: Sample larger than population or is negative

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing any data, which means that it was unable to verify the conjecture's conditions. | next: Re-run the test ensuring it completes without crashing and produces the required data for analysis.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	16026
2	preregistration	ollama_remote	glm4:latest	0	0	9412
3	novelty	ollama_remote	glm4:latest	0	0	8749
4	novelty	ollama_remote	glm4:latest	0	0	9320
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14071
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11486
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9193
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10266
9	judge	ollama_remote	glm4:latest	0	0	13476

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 101999 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/1e09a50da5e3.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/1e09a50da5e3.jsonl` for verification of provenance

4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/1e09a50da5e3.tar.gz` (if generated)